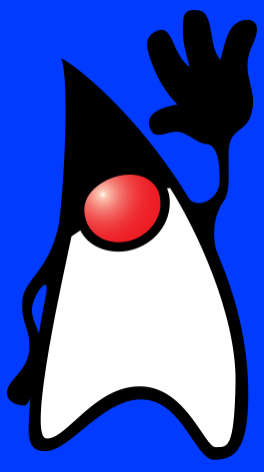
{EPITECH}

# JAVA SEMINAR_

< DAY 01 - HELLO DUKE />



# JAVA SEMINAR



For this bootcamp, you need Java to be installed on your computer.
We are here talking about **Java Standard Edition**, or JSE, that you could easily find on Internet.

In Java, every line of code must be included into a class.
There must also be one file by class (and one class by file) named the same way.
Here is a very first example of Java code (necessary in a file name ”HelloDuke.java”):

```java
public class HelloDuke {
    public static void main(String[] args) {
          System.out.println("Hello Duke!");
    }
}
```

Then this code is compiled with

```
javac HelloDuke.java
```

and executed:

```
java HelloDuke
Hello Duke!
```



Have you seen the ”main” function? Why is it special?



Research and identify on your own the common best practices for this language.
Optionally, extend your preferred IDE with tools for Java formatting and linting.



{EPITECH}

For all the following exercises, your functions must be included in a class named the same way as the file to turn-in. For instance, if the file to turn-in is `Ex0X.java`, the class must be:

```
class Ex0X {
      public static void myMethod(Type1 param1, Type2 param2, ...) {
      }
}
```

In the autograder, a testing class containing a `main` method will call Ex0X.myMethod as follows:

```
class TestMain {
      public static void main(String[] args) {
          Ex0X.myMethod(args[0], args[1], ...);
      }
}
```

This testing class will be called by the autograder as follows:

```
java TestMain value1 value2 ... // calling Ex0X.myMethod(value1, value2...);
THE GOOD RESULT WILL APPEAR HERE (hopefully!)
```

For each exercise, only commit your `.java` file, with required class and prototype.
You can test it locally, with your home made testing class and main, but do not commit it.

**Autograder says:**
When the required delivery is a specific file, with a specific file name, in a specific directory, with specific prototype, parameters, return type and restrictions, you deliver the right file with the right file name, in the right directory, with the right prototype, parameters, return type and restrictions. Period. Or else the test will fail.

**Autograder says (again):**
Do not ever commit `.class` files. Only `.java` files.

**Autograder says (finally):**
Do not put your classes into packages. At least, not unless you are asked to. If your IDE writes them automatically, remove them. We will review packages later this week.

## Exercise 01

**Delivery**: `./ex_01/Ex01.java`
**Restrictions**: `System.out.print` is mandatory, and must be used only once.
**Prototype**: `public static void myConcat(String str1, String str2);`

Create a `myConcat` function that takes two parameters and displays the first parameter followed by a space followed by the second parameter.

```
// calling Ex01.myConcat("Hello", "Duke");
Hello Duke
```





## Exercise 02

**Delivery**: `./ex_02/Ex02.java`
**Restrictions**: the `for` keyword is mandatory
**Prototype**: `public static String getAngryDog(int nbr);`

Write a function that takes a `nbr` integer as parameter and returns `nbr` times the word ”woof”, followed by a newline.

```
// calling Ex02.getAngryDog(3);
woofwoofwoof
```



{EPITECH}

## Exercise 03

**Delivery**: `./ex_03/Ex03.java`
**Restrictions**: one (and ONLY ONE) `for` keyword is mandatory. Beware your code AND your comments.
**Prototype**: `public static void printArray(ArrayList<String> myArray);`

Write a function that takes a `myArray` ArrayList as parameter and displays each of its value followed by a newline.

```
// calling Ex03.printArray([Volvo, BMW, Ford, Mazda]);
Volvo
BMW
Ford
Mazda
```



{EPITECH}

## Exercise 04

**Delivery**: `./ex_04/Ex04.java`
**Restrictions**: the `switch` keyword is mandatory. Plus, a new line MUST be printed after each display.
**Prototype**: `public static void printMovieFromNbr(int nbr);`

Write a function that takes an integer as parameter and displays

- ✓ `The Three Brothers` when the value is 3;
- ✓ `The Sixth Sense` when the value is 6;
- ✓ `The Number 23` when the value is 23;
- ✓ `28 Days Later` when the value is 28;
- ✓ `I don't know.` otherwise.

```
// calling Ex04.printMovieFromNbr(23);
The Number 23
```



{EPITECH}

## Exercise 05

**Delivery**: `./ex_05/Ex05.java`
**Prototype**: `public static ArrayList<String> myGetArgs(String... var);`

Write a function that takes as parameter a variable number of arguments and returns these arguments in an array.



{EPITECH}

## Exercise 06

**Delivery** `./ex_06/Ex06.java`
**Prototype**: `public static void sequence(int nbr);`

Write a function that outputs the following sequence to the $n^{th}$ iteration.
Here is the beginning of the sequence:

1
11
21
1211
111221
312211



The first iteration is 0.
Don't print anything when the number is negative.

```
// calling Exo06.sequence(0);
1
```

```
// calling Exo06.sequence(3);
1
11
21
1211
```



**Autograder says:** When we ask you to put a `,` or a `.` or a `, ` (comma, then space), you write them in your code. Period. Or else the test will fail. This warning stands for each and every autograded exercise of each and every module.



{EPITECH}

v 3.1.1

